pillar-UDP: Clustered Multipath Congestion Avoidance and Redundant Delivery over Lossy Networks

pillar engineering (machine-checked design note)

2026-08-31

Abstract

TCP and QUIC treat packet loss as the signal to *slow down*: a single path, a congestion window, additive-increase/multiplicative-decrease backoff. On a link where loss is the link's own fault—radio, satellite, jamming, cell-tower handoff—that reflex collapses throughput to near zero. pillar-UDP takes the opposite posture. Its unit of congestion control is not a window on one path but *the whole cell*: a message is delivered by spraying a *bounded* number of redundant, content-addressed datagrams across a topologically dispersed set of nodes and letting the receiver deduplicate by CID. Loss and latency are absorbed by redundancy; congestion is avoided by *finding a good route*, not by throttling. We give two TLA$^+$ models—a one-to-many non-node client and a many-to-many inter-cell full mesh—each with multiple nodes and multiple links, at least one adversarially lossy, and machine-check with TLC that (i) delivery always *routes around* the lossy links whenever the cell stays reachable by any dispersed path (liveness), (ii) redundant copies arriving at multiple nodes are accepted exactly once (CID dedup, safety), and (iii) per-round fan-out never exceeds the configured redundancy allowance (safety). We contrast this with the single-client, single-path congestion posture of TCP and QUIC. The redundancy header format and the quantitative scaling of the loss/bandwidth trade are the subject of a companion paper.

1 The problem: loss is not always congestion

The two guarantees of TCP—reliable, ordered byte delivery—are enforced by windows that regulate in-flight data, and those windows are a feedback loop tuned by observed loss. The design assumes loss *means* congestion, so the polite response is to back off. On a marginal link the assumption is simply false: loss there is the medium degrading, and backing off makes a recoverable link useless. The empirical cliff is steep—on a 20 ms path the first 1% of loss already removes about 80% of TCP throughput, and past roughly 17% loss TCP ceases to function at all; higher round-trip time makes recovery from each loss event dramatically slower [1]. QUIC modernises loss recovery and removes head-of-line blocking across streams, but its congestion posture is the same family: one path, one connection, a loss-driven congestion controller that throttles under loss.

pillar's deployments are exactly where this bites: cells span multiple sites on distinct provider prefixes, clients reach the cell over the open internet or worse, and inter-cell links cross congested or adversarial transit. pillar-UDP is the protocol for that regime.

2 Posture: redundancy plus clustered multipath routing

pillar-UDP's design posture is twofold, and each half maps onto any protocol that supports a subset of its features:

1. **Variable loss/latency tolerance through redundancy.** A message is a small, PGP-signed, content-addressed (CID) unit. The sender emits redundant copies; the receiver deduplicates by CID, so at-least-once transport plus CID identity yields exactly-once *processing*. Fan-out per round is capped by a *redundancy allowance* carried in the header, so a known-bad link states its characteristics up front.
2. **Automatic, dynamic, clustered multipath congestion avoidance.** The control variable is *route and source selection*, not window size. Redundant datagrams are emitted from as dispersed a set of node addresses as possible—steered by topology tier, GeoIP, and measured latency/loss—to maximise the chance that some path around the bad link is good.

Because trust is carried by the signature on each message and not by its origin address, the node that *replies* need not be the node that *ingested*: a cell can answer from whichever dispersed nodes have the best paths to the client. End-to-end IPv6 addressing makes this native; IPv4 behind NAT degrades it.

3 Formal models and what TLC proves

We model pillar-UDP at the level at which its guarantees live—delivery, deduplication, and bounded redundancy under an adversarial lossy channel—in two TLA$^+$ specifications, following the lossy-channel plus strong-fairness idiom pillar already uses for anti-entropy replication.

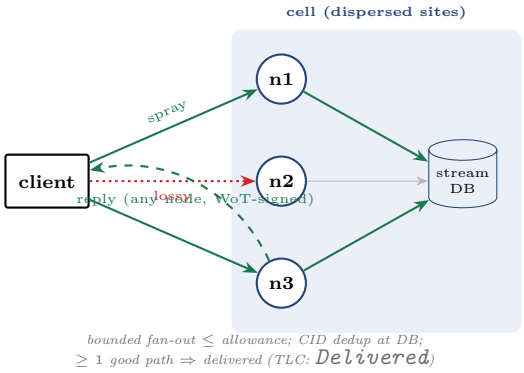

Figure 1: One-to-many (ingress class 2). A non-node client sprays redundant datagrams to a dispersed set of ingest nodes. The path to `n2` is lossy; copies still reach `n1` and `n3`, the streaming DB deduplicates by CID, and the reply returns from a *different* node than ingested it. Modeled in `PillarUdpClient.tla`.

Both make the pending work *monotone* so that an adversary cannot destroy in-flight progress, and both encode the route-around premise as a standing constraint on the adversary: it may flip *any* links lossy at any time, *provided the cell stays reachable by at least one dispersed path*. The theorem in each case is that pillar-UDP always finds that path, with no window control.

One-to-many (`PillarUdpClient`). A client sprays subsets of dispersed nodes (each spray bounded by the allowance; coverage accumulates monotonically across retransmit rounds). A copy reaches any node whose path is currently good; the first copy anywhere injects the message, later copies dedup. TLC checks:

- `ExactlyOnce` (safety): the message is injected at most once, though `received` may grow to several nodes.
- `SprayWidthBounded` (safety): no round exceeds the allowance.
- `Delivered` (liveness, ◇): the message is eventually injected despite arbitrarily many loss bursts.

Many-to-many (`PillarUdpMesh`). Two cells, each at several sites, form a full mesh of site-to-site links; a flow of messages is spread across the mesh. Any link may be lossy; delivery of each message counts once (CID dedup) regardless of how many sites redundantly forward it. TLC checks `ExactlyOnce` (safety) and `MeshCompletes` (liveness, ◇): the entire flow completes by routing around lossy links over the surviving site pairs.

Table 1 reports the actual TLC output. These are *logical* guarantees—exactly-once delivery and route-around progress hold over every interleaving of spraying, delivery, and adversarial loss. They are not throughput measurements; the quantitative loss/bandwidth trade is analytic (§5) and is treated fully in the companion paper.

model	distinct states	depth	result
`PillarUdpClient`	329	7	no error
`PillarUdpMesh`	3 840	10	no error

Table 1: TLC exhaustive model-checking results (deadlock checking on). Every listed invariant and liveness property holds over the complete state space of each finite instance. Both specs run in pillar's `specs/check.sh` gate.

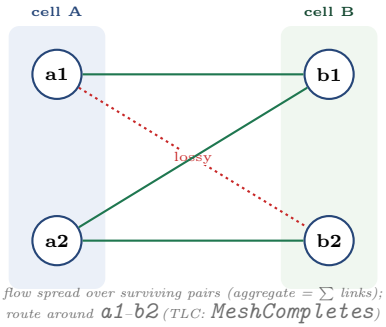

Figure 2: Many-to-many (ingress class 3). Two cells present at multiple sites form a full mesh. The `a1`–`b2` link is lossy; the A→B flow completes over the other three pairs, aggregating their bandwidth instead of bottlenecking on one connection. Modeled in `PillarUdpMesh.tla`.

4 Automatic geographic load balancing

Neither model contains a load balancer, a scheduler, or a route table. Delivery happens over *whatever dispersed path is currently good*; that is the load balancing. Because pillar-ipam binds addresses to topology labels (region, rack, node) and sites hold distinct prefixes, "spray from a dispersed set of IPs" is a topology-diversity query: diverse tiers give diverse prefixes give diverse paths. A node processes a message it can serve locally and forwards it under load; work therefore migrates to where capacity and good paths are, with near-zero configuration. The safety obligation that keeps this sound—that only idempotent messages may be processed opportunistically, while exclusive effects route to the coordination-core lease holder—is orthogonal to the transport and is modeled elsewhere; here every message is idempotent (CID dedup), so `ExactlyOnce` is exactly the property that opportunistic, redundant, multi-node delivery must not violate, and TLC confirms it does not.

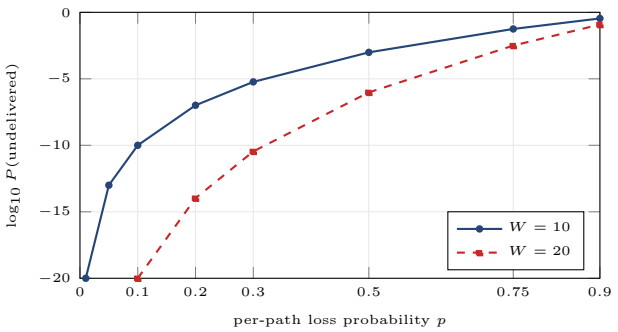

Figure 3: Residual loss p^W vs. per-path loss for redundancy windows W=10 and W=20 (points from the p^W table; dispersed paths make the independence assumption realistic). Lower is better; note the log scale.

5 Redundancy characteristics

Redundancy is what buys loss tolerance. If a message rides in W consecutive or parallel copies over paths with independent loss probability p, it is lost only if all W copies drop:

$$P(\text{undelivered}) = p^W.$$

Crucially, pillar-UDP's dispersion makes the independence assumption *hold*: copies sent from geographically and AS-diverse nodes do not share a bursty bottleneck, unlike W copies on one link. Figure 3 plots p^W: at 30% loss, W=10 already drives residual loss below 10^{-5}, and W=20 below 10^{-10}. The cost is bandwidth—fan-out scales with W—which is why the allowance is a per-connection budget and why the detailed loss/bandwidth scaling and header encoding are a separate paper. The model here proves the *qualitative* guarantee that redundancy plus a good path always completes; the figure shows the *rate* at which redundancy buys reliability.

6 Contrast with TCP and QUIC

Table 2 summarises the posture difference. The essential point is architectural, not parametric: TCP and QUIC place congestion control on a single client over a single path and react to loss by *reducing* the offered rate; pillar-UDP places it on a cluster over many paths and reacts to loss by *relocating* the offered traffic. On a healthy link QUIC is the right tool and remains pillar's default; pillar-UDP is selected—as an additional libp2p transport—only where the link itself is the problem, the regime in which single-path congestion control degrades toward zero.

	TCP / QUIC	pillar-UDP
unit of control	one connection, one path	the cell: many nodes, many paths
loss reaction	shrink congestion window	re-route / re-disperse redundancy
reliability via	retransmit after RTT	redundant copies (p^W) + NACK
identity / trust	5-tuple / TLS cert	per-message WoT signature (CID)
reply source	same connection	any dispersed node
lossy-link limit	collapses past ∼17% loss [1]	delivers while ≥ 1 path is good

Table 2: Congestion-handling posture. QUIC improves loss recovery and stream multiplexing over TCP but keeps the single-path, window-throttled posture.

7 Scope and limitations

The models are finite instances (three ingest nodes; a 2×2 mesh); they establish the protocol's logical guarantees, not a performance bound. The redundancy posture is deliberately *not* congestion-controlled in the TCP sense: it trades bandwidth for reliability and must therefore be gated to degraded/adversarial links and bounded by the allowance, never run as an open-internet default. Bulk transfer is a distinct problem addressed by erasure-coded, CID-verified chunking over the same dispersed spray, and the redundancy-header format and its loss/bandwidth scaling are the subject of the companion paper. The anti-amplification obligation (return-routability before committing redundant replies) and the idempotent-vs-exclusive processing classification are specified in their own models.

8 Conclusion

pillar-UDP replaces "find the right window" with "find the right route, and spray enough copies to survive the ones that fail." Two machine-checked TLA$^+$ models show that this delivers exactly once and always routes around lossy links as long as the cell stays reachable by any dispersed path—for a non-node client (one-to-many) and for an inter-cell full mesh (many-to-many) alike. The result is automatic geographic load balancing and aggregate-bandwidth cell-to-cell transport with near-zero configuration, in precisely the lossy, high-latency regime where single-path congestion control fails.

References

[1] G. Head, *Reliable Delivery Over Horrible Networks*, 2026. `https://blog.graysonhead.net/posts/reliable-delivery/`

[2] M. Frohnmayer and T. Gift, *The TRIBES Engine Networking Model*, 2000.

[3] L. Lamport, *Specifying Systems*, Addison-Wesley, 2002.